

Data 236- Distributed Systems

Homework-9

Kruthika Virupakshappa

Part 1

You have a blog where each post has a view counter. Every time someone reads a post, the view count increases. Querying the database for every view is slow. Use Redis to make it faster!

1. Increment the view count in Redis using the key "views:post:{post_id}"
 - Use Redis's INCR command: self.redis_client.incr(key)

```
def increment_post_views(self, post_id: int) -> int:
    """Increment view count in Redis; sync to database every 10 views"""
    key = f"views:post:{post_id}"
    view_count = self.redis_client.incr(key)

    if view_count % 10 == 0:
        self.cursor.execute("UPDATE users SET views = ? WHERE id = ?", (view_count, post_id))
        self.conn.commit()
        print("✓ Synced views to database")

    return view_count
```

2. Every 10 views , save the count back to the database
 - Check if view_count % 10 == 0
 - Update the database: UPDATE users SET views = ? WHERE id = ?
 - Print a message: "Synced views to database"

```
if view_count % 10 == 0:
    self.cursor.execute("UPDATE users SET views = ? WHERE id = ?", (view_count, post_id))
    self.conn.commit()
    print("✓ Synced views to database")
```

3. Return the current view count

```
return view_count
```

```
=====
== Testing View Counter ==
=====
✓ Cleared all cache data
View #1: Total views = 1
View #2: Total views = 2
View #3: Total views = 3
View #4: Total views = 4
View #5: Total views = 5
View #6: Total views = 6
View #7: Total views = 7
View #8: Total views = 8
View #9: Total views = 9
View #10: Total views = 10
✓ Synced views to database
View #11: Total views = 11
View #12: Total views = 12
View #13: Total views = 13
View #14: Total views = 14
View #15: Total views = 15
View #16: Total views = 16
View #17: Total views = 17
View #18: Total views = 18
View #19: Total views = 19
View #20: Total views = 20
✓ Synced views to database
View #21: Total views = 21
View #22: Total views = 22
View #23: Total views = 23
View #24: Total views = 24
View #25: Total views = 25
=====
```

Part 2: Semantic Caching with Redis and Ollama

Assignment: Build a semantic caching system that stores Ollama LLM responses in Redis using vector embeddings. Configure Redis with a vector index to store query embeddings (using a model like SentenceTransformers), set a similarity threshold around 0.85, and implement logic that generates an embedding for each incoming query, searches Redis for semantically similar cached queries using cosine similarity, and returns the cached response if the similarity score exceeds your threshold. When no similar query exists, send the request to Ollama, get the response, then store both the query embedding and response in Redis for future cache hits.

Workflow:

User query



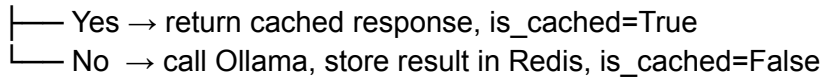
Generate embedding



Search Redis vector index



Is similarity ≥ 0.85 ?



Configuration:

```
REDIS_HOST      = "localhost"
REDIS_PORT      = 6379
VSET_KEY        = "semantic_cache"
EMBEDDING_MODEL = "all-MiniLM-L6-v2"
EMBEDDING_DIM   = 384
SIMILARITY_THRESH = 0.85
OLLAMA_MODEL    = "llama3.2:latest"
```

Redis and embedding model initialization:

```
def get_redis() -> redis.Redis:
    global _redis_client
    if _redis_client is None:
        _redis_client = redis.Redis(host=REDIS_HOST, port=REDIS_PORT,
                                     decode_responses=False)
        _redis_client.ping()
    return _redis_client

def get_encoder() -> SentenceTransformer:
    global _encoder
    if _encoder is None:
        log.info("Loading embedding model '%s' ...", EMBEDDING_MODEL)
        _encoder = SentenceTransformer(EMBEDDING_MODEL)
        log.info("Embedding model ready.")
    return _encoder
```

Embedding generation:

```
def embed(text: str) -> np.ndarray:
    return get_encoder().encode(text, normalize_embeddings=True).astype(np.float32)
```

Redis vector storage:

```
def vset_add(query: str, response: str, vec: np.ndarray) -> None:
    r = get_redis()
```

```

key = uuid.uuid4().hex
r.execute_command("VADD", VSET_KEY, "FP32", vec.tobytes(), key)
r.execute_command("VSETATTR", VSET_KEY, key,
                  json.dumps({"query": query[:500], "response": response}))

```

Redis semantic search:

```

def vset_search(vec: np.ndarray) -> tuple[Optional[str], float, str]:
    r = get_redis()
    raw = r.execute_command(
        "VSIM", VSET_KEY, "FP32", vec.tobytes(), "COUNT", 1, "WITHSCORES"
    )
    if not raw:
        return None, 0.0, ""
    element = raw[0].decode()
    similarity = float(raw[1])
    attr_raw = r.execute_command("VGETATTR", VSET_KEY, element)
    if attr_raw is None:
        return None, similarity, ""
    attrs = json.loads(attr_raw.decode())
    if similarity >= SIMILARITY_THRESH:
        return attrs.get("response", ""), similarity, attrs.get("query", "")
    return None, similarity, attrs.get("query", "")

```

Core query-handling logic:

```

vec = embed(query)

if cache_size == 0:
    response, similarity, matched_query = None, 0.0, ""
else:
    response, similarity, matched_query = vset_search(vec)

is_cached = response is not None
_stats["total"] += 1

if is_cached:
    _stats["hits"] += 1
    total_ms = (time.perf_counter() - t0) * 1000
    _stats["total_cached_ms"] += total_ms
else:
    _stats["misses"] += 1
    response = ollama.chat(
        model=OLLAMA_MODEL,
        messages=[{"role": "user", "content": query}],
    )["message"]["content"].strip()

    vset_add(query, response, vec)
    total_ms = (time.perf_counter() - t0) * 1000
    _stats["total_live_ms"] += total_ms

```

Output fields showing cache tracking:

```
yield sse("result", {
    "response": response,
    "is_cached": True,
    "similarity": round(similarity, 4),
    "total_ms": round(total_ms, 1),
})
```

```
yield sse("result", {
    "response": response,
    "is_cached": False,
    "similarity": round(similarity, 4),
    "llm_ms": round(llm_ms, 1),
    "total_ms": round(total_ms, 1),
})
```

Performance metrics snippet:

```
t = _stats["total"]
h = _stats["hits"]
m = _stats["misses"]
avg_cached_ms = _stats["total_cached_ms"] / h if h else 0
avg_live_ms = _stats["total_live_ms"] / m if m else 0
speedup = avg_live_ms / avg_cached_ms if avg_cached_ms else 0
```

The screenshot shows the Semantic Cache web application interface. At the top, the title "Semantic Cache" is displayed in a large, bold font. Below the title, a subtitle reads "Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE". A row of five status tags is shown: "Redis VectorSet", "all-MiniLM-L6-v2", "Ollama llama3.2", "Cosine ≥ 0.85", and "SSE Streaming". Below these tags, four performance metrics are displayed in separate boxes: "0 QUERIES", "0% HIT RATE", "— AVG CACHE MS", and "— AVG OLLAMA MS". A large text input area follows, with the placeholder text "Ask anything — try the same question twice, or rephrase it...". Below the input area, a small note states "Exact or paraphrased queries hit the cache · New queries call Ollama". To the right of the input area are two buttons: "Flush Cache" and "Ask". At the bottom of the interface, a footer line reads "HW9 · Data 236 Distributed Systems · Semantic Caching with Redis + Ollama".

Outputs: Test the system with 10 diverse queries including exact duplicates, paraphrased questions, and completely new queries to evaluate performance.

1. Cache Miss (first query)- Ollama Live badge

The screenshot displays the Semantic Cache application interface. At the top, the title "Semantic Cache" is shown in white and green. Below it, the stack of technologies is listed: "Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE". Five status badges are present: "Redis VectorSet", "all-MiniLM-L6-v2", "Ollama llama3.2", "Cosine ≥ 0.85", and "SSE Streaming". A performance summary row shows: 1 QUERY, 0.0% HIT RATE, — AVG CACHE MS, and 13398 ms AVG OLLAMA MS. The main query input area contains "What is machine learning?". Below the input, a note states "Exact or paraphrased queries hit the cache · New queries call Ollama". Action buttons for "Flush Cache" and "Ask" are visible. The response area shows the query "What is machine learning?" with a blue "OLLAMA LIVE" badge and a response time of "13398 ms". The response text is: "Machine learning (ML) is a subset of artificial intelligence (AI) that enables systems to automatically improve their performance on a task without being explicitly programmed. It involves training algorithms on data, so they can make predictions or take actions based on new, unseen data. The key characteristics of machine learning are:".

Semantic Cache

Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet all-MiniLM-L6-v2 Ollama llama3.2 Cosine ≥ 0.85 SSE Streaming

1
QUERIES

0.0%
HIT RATE

—
AVG CACHE MS

13398 ms
AVG OLLAMA MS

What is machine learning?

Exact or paraphrased queries hit the cache · New queries call Ollama

Flush Cache Ask

What is machine learning? **OLLAMA LIVE** 13398 ms

Machine learning (ML) is a subset of artificial intelligence (AI) that enables systems to automatically improve their performance on a task without being explicitly programmed. It involves training algorithms on data, so they can make predictions or take actions based on new, unseen data.

The key characteristics of machine learning are:

2. Exact Duplicate query: What is machine learning?
Cache Hit badge with Similarity 100% and response time 68.6ms (dramatic speedup)

Semantic Cache

Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet

all-MiniLM-L6-v2

Ollama llama3.2

Cosine ≥ 0.85

SSE Streaming

2

QUERIES

50.0%

HIT RATE

77 ms

AVG CACHE MS

13398 ms

AVG OLLAMA MS

What is machine learning?

Exact or paraphrased queries hit the cache · New queries call Ollama

Flush Cache

Ask

What is machine learning?

CACHE HITsim 100.0%77 ms

Machine learning (ML) is a subset of artificial intelligence (AI) that enables systems to automatically improve their performance on a task without being explicitly programmed. It involves training algorithms on data, so they can make predictions or take actions based on new, unseen data.

The key characteristics of machine learning are:

3. Can you explain machine learning?
Cache Hit badge with similarity score 92.3%

Semantic Cache

Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet

all-MiniLM-L6-v2

Ollama llama3.2

Cosine ≥ 0.85

SSE Streaming

3

QUERIES

66.7%

HIT RATE

115.3 ms

AVG CACHE MS

13398 ms

AVG OLLAMA MS

Can you explain machine learning?

Exact or paraphrased queries hit the cache · New queries call Ollama

Flush Cache

Ask

Can you explain machine learning?

CACHE HIT

sim 92.3%

153.7 ms

Machine learning (ML) is a subset of artificial intelligence (AI) that enables systems to automatically improve their performance on a task without being explicitly programmed. It involves training algorithms on data, so they can make predictions or take actions based on new, unseen data.

The key characteristics of machine learning are:

4. How does photosynthesis work?
Different topic

Semantic Cache

Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet

all-MiniLM-L6-v2

Ollama llama3.2

Cosine ≥ 0.85

SSE Streaming

4

QUERIES

50.0%

HIT RATE

115.3 ms

AVG CACHE MS

14011.2 ms

AVG OLLAMA MS

How does photosynthesis work?

Exact or paraphrased queries hit the cache · New queries call Ollama

Flush Cache

Ask

How does photosynthesis work?

OLLAMA LIVE

14624.5 ms

Photosynthesis is a fascinating process that occurs in plants, algae, and some bacteria, where they convert light energy from the sun into chemical energy in the form of glucose. This process is essential for life on Earth, as it provides the primary source of energy for nearly all living organisms.

Here's a step-by-step explanation of how photosynthesis works:

5. Explain the process of photosynthesis

Similarity score 94.5% with cache hit badge

Semantic Cache

Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet

all-MiniLM-L6-v2

Ollama llama3.2

Cosine ≥ 0.85

SSE Streaming

5

QUERIES

60.0%

HIT RATE

100.5 ms

AVG CACHE MS

14011.2 ms

AVG OLLAMA MS

Explain the process of photosynthesis

Exact or paraphrased queries hit the cache · New queries call Ollama

Flush Cache

Ask

Explain the process of photosynthesis

CACHE HIT

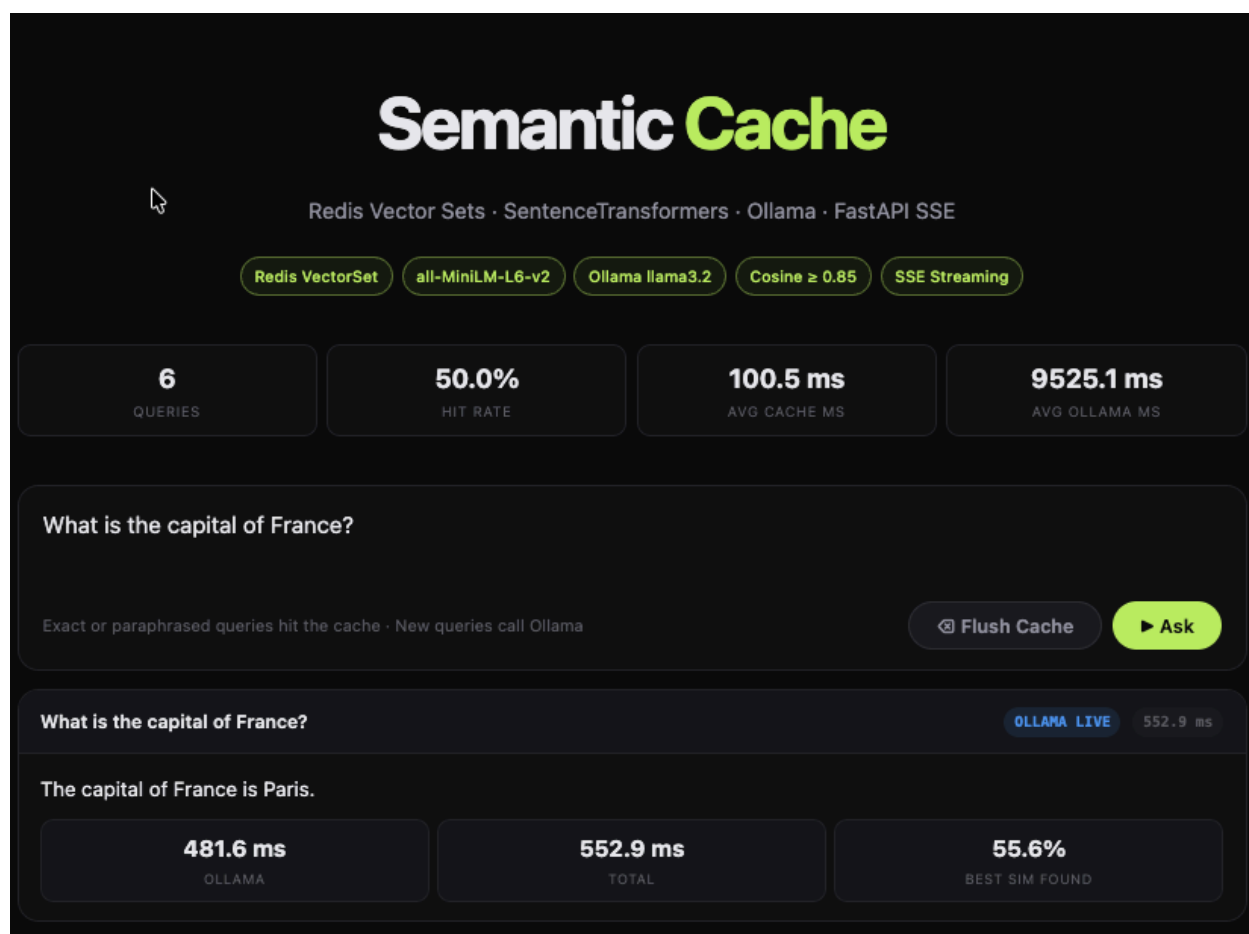
sim 94.5%

70.9 ms

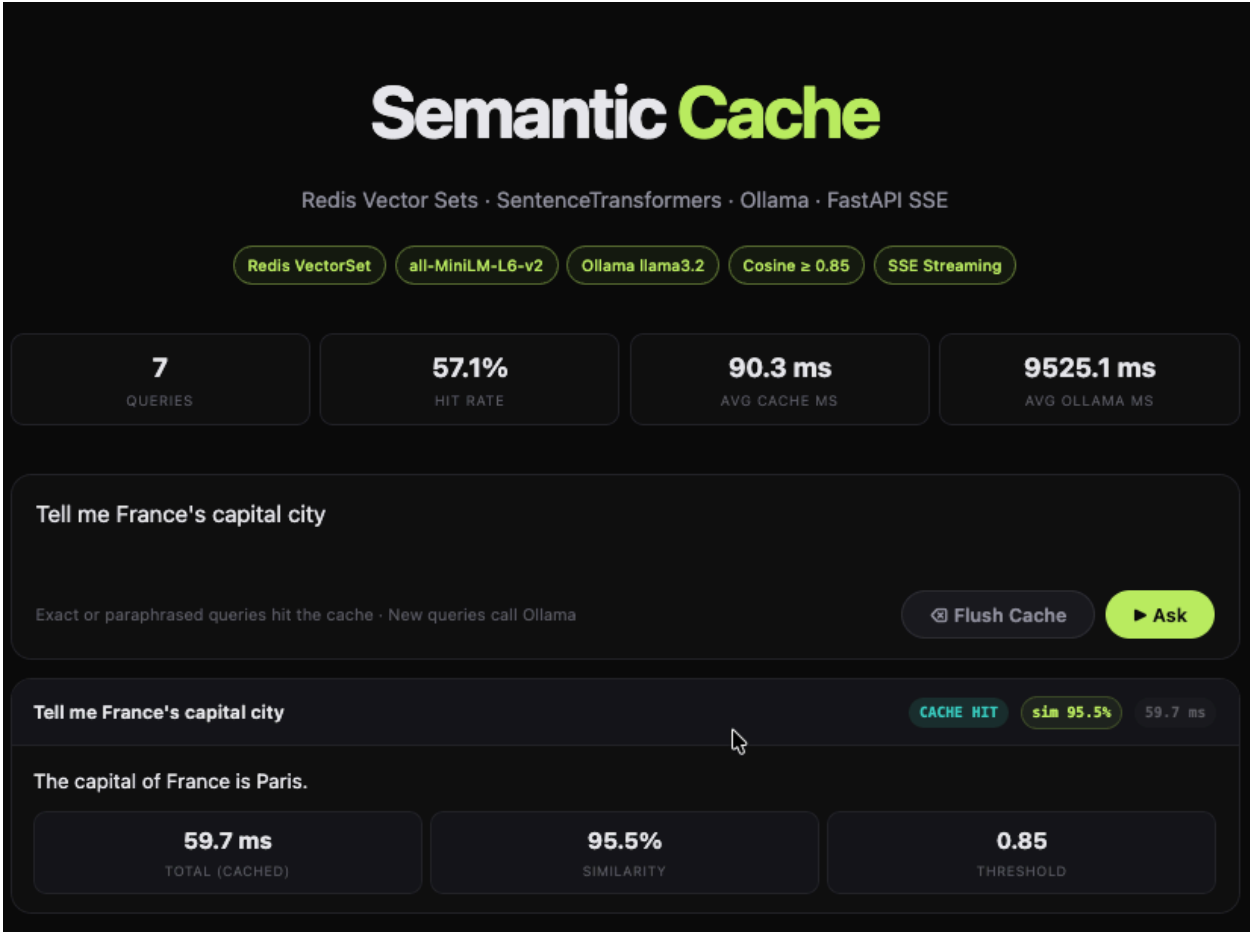
Photosynthesis is a fascinating process that occurs in plants, algae, and some bacteria, where they convert light energy from the sun into chemical energy in the form of glucose. This process is essential for life on Earth, as it provides the primary source of energy for nearly all living organisms.

Here's a step-by-step explanation of how photosynthesis works:

6.What is the capital of France?
Cache miss



7. Tell me France's capital city
Cache hit with 95.5% similarity score



8. How do I brew a good coffee?
Cache miss

Semantic Cache

Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet

all-MiniLM-L6-v2

Ollama llama3.2

Cosine ≥ 0.85

SSE Streaming

8

QUERIES

50.0%

HIT RATE

90.3 ms

AVG CACHE MS

10882.8 ms

AVG OLLAMA MS

How do I brew a good coffee?

Exact or paraphrased queries hit the cache · New queries call Ollama

⌂ Flush Cache

▶ Ask

How do I brew a good coffee?

OLLAMA LIT

14955.7 ms

Brewing a good cup of coffee requires attention to detail and a bit of practice. Here's a step-by-step guide to help you get started:

****Materials needed:****

9. What is the best way to make coffee?
Cache hit with 90.8% similarity score

Semantic Cache

Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet all-MiniLM-L6-v2 Ollama llama3.2 Cosine ≥ 0.85 SSE Streaming

9 QUERIES	55.6% HIT RATE	88 ms AVG CACHE MS	10882.8 ms AVG OLLAMA MS
--------------	-------------------	-----------------------	-----------------------------

What is the best way to make coffee?

Exact or paraphrased queries hit the cache · New queries call Ollama

Flush Cache Ask

What is the best way to make coffee? CACHE HIT sim 90.8% 78.9 ms

Brewing a good cup of coffee requires attention to detail and a bit of practice. Here's a step-by-step guide to help you get started:

****Materials needed:****

10. What is quantum computing?
Cache miss

Semantic Cache



Redis Vector Sets · SentenceTransformers · Ollama · FastAPI SSE

Redis VectorSet all-MiniLM-L6-v2 Ollama llama3.2 Cosine ≥ 0.85 SSE Streaming

10 QUERIES	50.0% HIT RATE	88 ms AVG CACHE MS	11061.1 ms AVG OLLAMA MS
---------------	-------------------	-----------------------	-----------------------------

What is quantum computing?

Exact or paraphrased queries hit the cache · New queries call Ollama

⌛ Flush Cache ▶ Ask

What is quantum computing?

OLLAMA LIVE 11774.5 ms

Quantum computing is a type of computer that uses the principles of quantum mechanics to perform calculations and operations on data. It's a new paradigm for computing that has the potential to revolutionize many fields, including cryptography, optimization, and simulation.

Classical computers use bits to store and process information, which can only be in one of two states: 0 or 1. Quantum